

SYSTEMD

Helix Platform — systemd Boot Integration

Revision: 4 **Last modified:** 2026-09-03 **Maintainer:** HelixCode platform

The whole Helix platform — HelixCode, HelixAgent, HelixLLM and their infrastructure — is installed as systemd **user** units that start on host boot and survive restarts.

Quick reference

```
./setup.sh                                # install everything
    (units included)
./setup.sh --start                         # ... and start it
    now

systemctl --user start    helix.target    # bring the whole
    platform up
systemctl --user stop    helix.target    # bring it all down
systemctl --user status  helixagent      # one service
systemctl --user list-units 'helix*'     # all of them
journalctl --user -u helixagent -f       # follow one
    service's logs

scripts/install_systemd_units.sh          # re-install after
    editing a unit
scripts/install_systemd_units.sh --start  # install + enable +
    start
scripts/install_systemd_units.sh --uninstall

# Operator wrapper (recommended)
scripts/helixctl.sh install              # build + install +
    start
scripts/helixctl.sh start|stop|restart
scripts/helixctl.sh status [unit]
scripts/helixctl.sh enable|disable
scripts/helixctl.sh validate             # run platform
    readiness probe
scripts/helixctl.sh logs <unit>
```

The `helixctl.sh` wrapper is the single operator entry point: every action is implemented through `systemctl --user`, so the whole platform is controlled via `systemd` user-space. `validate` runs `scripts/validate-helix-platform.sh`, which probes every unit and every exposed port and writes a timestamped report under `qa-results/helix-platform-validation/`.

Prove the platform is actually *serving*, not merely *active* (§11.4.108):

```
curl -s      http://localhost:8100/api/scores      #
           llmsverifier
curl -sk     https://localhost:8443/internal/health # helixllm-
           gateway (TLS)
curl -s      http://localhost:7061/health          # helixagent
curl -s      http://localhost:8080/health          # helixcode-
           server
```

Port note: helixcode-server serves :8080 (corrected revision 4)

Revision 3 of this document recorded `:8081`. The unit now serves **:8080**, which is the port `helix_code/config/config.yaml` declares (line 6) and the port the running service actually binds. `config/replica-8081.yaml` is — as its name says — a `SECOND` instance’s config; its `:8081` and its `:5433/:6380` datastore stanzas address the `helixcode-infra-*` stack, which is not enabled in this deployment. `helixcode-server.service` therefore sets `HELIX_CONFIG=config/config.yaml` explicitly.

Setting it explicitly is load-bearing, not tidiness: the binary’s fallback chain ends at `~/.config/helixcode/config.json`, and that file currently carries ~75 keys this build rejects, so a start from any directory without a cwd-relative `config/config.yaml` aborts with `Failed to load configuration: unknown configuration`.

Units

`helix.target` is the umbrella; each service declares `WantedBy=helix.target`, so enabling a service wires it in automatically.

Unit	Type	Port	Purpose
<code>helix.target</code>	target	—	Umbrella — start/stop the platform as one

Unit	Type	Port	Purpose
helixcode- infra.service	oneshot	see below	11 containers via podman compose — Postgres, Redis, memcached, Ollama, Selenium, chromedp, Weaviate, ChromaDB, Qdrant, Cognee, plus a multicast- router sidecar that publishes no host port
llmsverifier.service	simple	8100	LLMsVerifier — model/provider verification + scoring API
helixllm- coder.service	oneshot	18434	Local Qwen3- Coder-30B model container
helixllm- gateway.service	simple	8443 (TLS)	HelixLLM multi- provider LLM router
helixagent.service	simple	7061	HelixAgent runtime (HTTP/ 1.1+2 on TCP, HTTP/3 on UDP)
helixcode- server.service	simple	8080	HelixCode API server

Start ordering: infra → llmsverifier → llm-coder / llm-gateway →
helixagent → helixcode-server.

Dependency strength is deliberate per service:

- helixagent uses Requires=helixcode-infra.service — its startup dependency verification is mandatory and hard-fails, so starting without infra is pointless.

- `helixllm-gateway` uses `Wants=llmsverifier.service` — the gateway polls `{verifier}/api/scores` and falls back to a static score table when it is unreachable. That fallback is a real working mode, so a down verifier must not prevent the gateway from starting and routing.
- `helixcode-server` uses `Wants=` for the same reason: it has its own infra-boot fallback path.

Live state captured 2026-07-28 — a point-in-time observation, not a standing guarantee; re-probe before relying on it. All six units active, enabled, `NRestarts=0`; 14 containers running on the host, 11 healthy, 0 unhealthy; each of the four HTTP health routes in the Quick reference returned 200.

Full stop/start cycle executed 2026-08-06 (evidence: `docs/qa/live_systemd_boot_20260806T145857Z/`) — the first pass in which the platform was actually torn down and cold-booted rather than observed running. Teardown was clean (all six inactive, none failed, 0 containers, all 18 documented ports closed). The cold boot converged to all six active, 12 containers, all 18 ports open, and all four health routes 200 — but **not cleanly**, and the two headline results below are only visible on a cold boot:

- `helixagent` **hard-failed once and finished with `NRestarts=1`** — the Cognee readiness race, now tracked as HXC-228 (see *Gotchas*).
- `helixllm-gateway` took **37 minutes** (`22:24:54→23:01:52`) from `ExecStart` to server listening `addr=0.0.0.0:8443`, reporting active (running) throughout — tracked as HXC-231.

A boot that is judged only after it has settled looks perfect; both defects are invisible unless the boot itself is watched.

Port map

Port	Service	Notes
8080	<code>helixcode-server</code>	<code>config/config.yaml</code> (primary). <code>config/replica-8081.yaml</code> is a SECOND instance's config, not the primary — see note below.
8443	<code>helixllm-gateway</code>	TLS — use <code>https://</code> . Health route is <code>/internal/health</code> (registered in <code>submodules/helix_llm/internal/server/server.go:167</code>). <i>Not</i> <code>/v1/models</code> — that is the path the gateway uses to probe downstream providers, and it answers

Port	Service	Notes
		{"object": "list", "data": null} until provider keys are configured, so it is a misleading health signal
7061	helixagent	TCP <i>and</i> UDP (QUIC)
8100	llmsverifier	the address the gateway looks for; the binary's own default is 8080, so the unit passes --port 8100 explicitly
18434	helixllm-coder	local model
5433	infra postgres	not 5432 — 5432 belongs to an unrelated stack on this host
6380	infra redis	not 6379, same reason
8083	infra weaviate	HTTP; moved off 8081 to avoid the replica config's port
50051	infra weaviate	gRPC
8082	infra chromadb	
8000	infra cognee	
6333/6334	infra qdrant	
11434	infra ollama	
11211	infra memcached	
4444/7900	infra selenium	
9222	infra chromedp	
—	infra multicast- router	sidecar; container-network only, publishes no host port

Boot chain

These are **user** units, not system units, because the services run rootless podman (§11.4.161) as the invoking user and read that user's ~/.config/cdi GPU specs.

Boot therefore depends on three links, **all** of which must hold:

```

loginctl Linger=yes                                # user manager runs without a l
└─ default.target.wants/helix.target                # helix.target enabled
    └─ helix.target.wants/*.service                 # each service enabled

```

1. **Linger.** User units normally start at *login*. `loginctl enable-linger $USER` is what makes them start at *boot* instead — the installer enables it and reports whether it succeeded. Without *linger* the platform only comes up once someone logs in. This is the link most often missing.
2. **helix.target enabled.** The target declares `WantedBy=default.target`, so enabling it creates the `default.target.wants/helix.target` symlink that pulls the platform in at boot.
3. **Each service enabled.** Every service declares `WantedBy=helix.target`, so `systemctl --user enable <unit>` creates the `helix.target.wants/<unit>` symlink. The target file itself enumerates no members — it is populated purely by these symlinks, which is why adding or removing a service never requires editing the target.

Enable / disable

enable everything (what the installer does)

```

systemctl --user enable helix.target
systemctl --user enable helixcode-infra helixllm-coder helixllm-
gateway \
llmsverifier helixagent helixcode-server
loginctl enable-linger "$USER"

```

disable one service – removes only its helix.target.wants/symlink

```
systemctl --user disable helixcode-server
```

stop the platform coming up at boot, leaving the units installed

```
systemctl --user disable helix.target
```

`enable` only wires a unit into the boot graph; it does not start it. Add `--now`, or `systemctl --user start helix.target`, to do both.

Verify all three links:

```

loginctl show-user "$USER" -p Linger                # expect
Linger=yes
ls ~/.config/systemd/user/default.target.wants/    # expect
helix.target
ls ~/.config/systemd/user/helix.target.wants/
# expect the six services
systemctl --user is-enabled helix.target helixagent

```

Unit templating

The units in `scripts/systemd/` are **templates** containing `@HELIX_ROOT@` and `@HELIXLLM_BIN@`. `scripts/install_systemd_units.sh` expands them for the current checkout when installing into `~/ .config/systemd/user/`.

Never hand-edit the installed copies — edit the template in `scripts/systemd/` and re-run the installer, otherwise the change is lost on the next install and a fresh clone cannot reproduce it. The installer refuses to install a unit that still contains an unexpanded placeholder.

Secrets

`helixagent` needs `DB_PASSWORD` and `JWT_SECRET`; both are read from `.env` via `EnvironmentFile=` and are **never** inlined into unit files, which are world-readable and version-controlled (CONST-042 / §12.1). `.env` is mode 0600 and `gitignored`.

If `.env` is missing, `EnvironmentFile=-` (leading `-`) makes it optional, so the service fails with a clear dependency error rather than a unit-file parse error.

Gotchas

A unit env var the binary never reads fails silently

These binaries are env-configured through `envconfig` struct tags, not through a config flag. If a unit sets a variable name the struct does not declare, the setting is simply **inert** — no parse error, no warning, no failed unit. The service starts, reports `active (running)`, and quietly runs on the compiled-in default. This is the worst failure shape in the whole file: everything is green and the intent never took effect.

Two instances of this class, both found and fixed on 2026-07-28 in `helixllm-gateway.service`:

Unit set	Binary actually reads	Consequence of the mismatch
HELIX_CACHE_REDIS_HOST / HELIX_CACHE_REDIS_PORT	HELIX_REDIS_HOST / HELIX_REDIS_PORT (<code>config.go:121-122</code> , port default 6379)	Redis fell back to :6379, where nothing listens

Unit set	Binary actually reads	Consequence of the mismatch
<i>(unset)</i>	HELIX_MODELS_DIR (config.go:70, default /models)	Default is a container path; running natively, every model auto-download failed

Symptoms the gateway logged on every start, before the fix:

```
dial tcp 127.0.0.1:6379: connect: connection refused
KV cache: Redis unreachable, falling back to in-memory
downloader: create models dir /models: mkdir /models: permission denied
```

Note that both fell back to a *working degraded mode* — in-memory cache, no downloads — which is precisely why they survived unnoticed.

Rule: when adding or changing an Environment= line, diff the name against the binary's actual env tags. Never infer the name from the config key.

```
# every env var the binary really reads
grep -rn 'env:"' submodules/helix_llm/internal/shared/config/
config.go
```

```
# every env var the unit sets
grep -o 'Environment=[A-Z_]*' scripts/systemd/helixllm-
gateway.service
```

Compiled-in defaults aimed at a container (/models, :5432, :6379, :8001) are a recurring source of this: they are plausible, they are wrong for a native user service, and they never announce themselves.

active does not mean *listening* — the gateway proves it

helixllm-gateway downloads models **before** it binds its listener. Between ExecStart and the first accepted connection the unit is already active (running) while :8443 refuses connections. On a first boot after a model-cache wipe that window is **several minutes** long.

Measured 2026-08-06: 37 minutes — ExecStart 22:24:54, server listening 23:01:52. A curl at 22:55 returned HTTP 000 (connection refused) while systemctl reported active (running) and ss -ltn showed nothing on 8443. The window is bounded by an *external* dependency (HuggingFace download speed), not by anything local: on that run the second model aborted after ~30 minutes with context deadline exceeded, and had the site been unreachable the gateway would arguably never have bound at all. Tracked as HXC-231.

So a green `systemctl --user status` during that window is accurate and useless. Probe the port, not the unit state (§11.4.108):

```
curl -sk https://localhost:8443/internal/health    # the real
                                                    readiness signal
journalctl --user -u helixllm-gateway -f          # watch the
                                                    download progress
```

Raising `TimeoutStartSec` does not help here: the unit is `Type=simple`, so `systemd` considers it started the moment the process forks — it is not waiting on readiness at all.

Every cold boot fails `helixagent` once — the `infra oneshot` lies about readiness

`helixcode-infra` is `Type=oneshot` running `podman compose up -d`, which returns when the containers have been **created**, not when they are **serving**. `helixagent` declares `Requires=/After=helixcode-infra.service`, so `systemd` considers the dependency satisfied and starts it seconds later — into a window where `Cognee`’s port is already forwarded by `rootlessport` but the app behind it is not yet accepting. `HelixAgent`’s dependency check is strict and fatal, so it exits 1:

```
DEPENDENCY FAILED dependency=Cognee
error="... read tcp 127.0.0.1:43606->127.0.0.1:8000: read: connection r
level=fatal msg="Application failed" ... BOOT BLOCKED: 1 of 4 mandatory de
helixagent.service: Failed with result 'exit-code'.
```

`Restart=` recovers it ~16 s later and the retry passes all four checks, so the settled state looks perfect. Reproduced identically on 2026-07-27 and on the 2026-08-06 cold boot. The tell is `NRestarts=1`:

```
systemctl --user show helixagent -p NRestarts    # expect 0; a
                                                    cold boot gives 1
```

This self-heals *by luck* — it depends on `Cognee` winning a race against the restart budget. Tracked as HXC-228; the fix is for the `infra` unit to wait for container health rather than for `compose up` to return.

`systemctl stop helix.target` returns long before the platform is down

All six services declare `PartOf=helix.target`, so stopping the target genuinely does propagate — the doc’s “bring it all down” is correct in outcome. But the propagated stops are **separate jobs that `systemctl` does not wait for**. On 2026-08-06 `systemctl --user stop helix.target` returned exit 0 in **11 ms**

while helixllm-coder and helixllm-gateway were still deactivating and 7 containers were still running; full quiescence took up to ~2 minutes.

Do not treat the command's exit as teardown-complete — poll for it:

```
systemctl --user stop helix.target
until [ "$(podman ps -q | wc -l)" -eq 0 ] \
    && ! systemctl --user is-active --quiet helixagent; do
    sleep 1; done
```

podman ps is not a reachability oracle

helixcode-infra.service sets KillMode=process. podman compose up -d returns immediately but leaves common and rootlessport (the host-side port forwarders) in the unit's cgroup. Under the default KillMode=control-group, any stop *or failure* makes systemd SIGKILL the whole cgroup — killing the port forwarders of every container in the stack while the containers themselves survive.

When that happens podman ps still reports Up (healthy) 0.0.0.0:5433->5432/tcp, because the container is alive; only the forwarder is gone. Nothing on the host can connect.

Always confirm reachability with ss -ltn or curl, never with podman ps.

```
# what is ACTUALLY reachable
for p in 5433 6380 8083 8082 8000 6333 11434; do
    ss -ltn | grep -q ":$p " && echo "ok $p" || echo "DEAD $p"
done
```

helixagent serves HTTP/3, so it binds UDP too

ss -ltn (TCP only) will not show the QUIC listener. Use ss -ltn *and* ss -lun, or ss -an | grep 7061.

Troubleshooting

systemctl --user status <unit> -l	# why it failed
journalctl --user -u <unit> -n 50	# recent log
journalctl --user -u <unit> --since today	# today's log
systemctl --user reset-failed	# clear failed state
before retrying	
systemctl --user cat <unit>	# the installed
(expanded) unit	

A unit is active (running) but nothing is listening. Unit state is not proof the service works (§11.4.108). Check the journal for where startup stalled, and probe the port directly. For helixllm-gateway this is expected behaviour during model download, not a fault — see *Gotchas*.

Infra fails with address already in use. Something else on the host holds the port. Identify the owner *before* acting — it may belong to another project (§11.4.174):

```
ss -ltnp | grep ':<port>'
```

Known gaps

Recorded honestly rather than papered over (§11.4.6):

- **distributed_locks and agent_instances tables have no schema.**

HelixAgent references distributed_locks from three Go files but no migration or .sql file defines it, so it logs Error cleaning locks:

ERROR: relation "distributed_locks" does not exist (SQLSTATE 42P01) every 60 s. **Re-verified 2026-08-06 directly against the databases rather than only from the log** — neither table exists in *any* of the three (postgres, helixcode_test, helixagent_db):

```
for db in postgres helixcode_test helixagent_db; do
    podman exec helixcode-infra-postgres psql -U helixcode -d
        "$db" \
        -tAc "select to_regclass('public.distributed_locks');"
        # → empty in all three
done
```

Rate is exactly 60 occurrences per 60 minutes. agent_instances is the same class, surfaced once during HelixAgent boot (relation "agent_instances" does not exist). Non-fatal — the service runs normally — but the locking feature these tables back is silently inert. Tracked as HXC-232. Still needs a migration authored by someone who can specify the intended columns; not invented here.

- **helixllm-gateway lists no models** — GET /v1/models returns {"object":"list","data":null} (re-confirmed 2026-08-06) until provider API keys are configured in .env. The API itself works; use /internal/health for readiness. Note this is now *doubly* explained: no provider keys **and** no local model server (next item).
- **llama-server is not installed on this host at all.** The gateway logs Failed to start embedded llama-server ... executable file not found

in `$PATH` on every start. Verified 2026-08-06 that this is **not** a `PATH` problem — no file named `llama-server` exists anywhere under `/usr/local/bin`, `/usr/bin`, `/bin`, `~/local/bin`, `/opt` or `$HOME`. The gateway nevertheless spends the 37-minute download window fetching a model for it, so the downloaded `.gguf` sits on disk unusable. Tracked as HXC-233.

- **HelixAgent's MCP plug-in servers fail to build and never start.** On each boot HelixAgent runs `podman-compose ... docker-compose.mcp-servers.yml up -d`, which fails at `RUN pnpm install ... && pnpm -r run build` (and the `@supabase/mcp-server-supabase` build) with exit status 1. HelixAgent downgrades this to `Failed to start some MCP servers (local mode)` and continues, so the platform looks healthy while every MCP-provided capability is absent. Observed twice in the 2026-08-06 boot; each attempt also re-pulls `node:22-bookworm-slim`, lengthening every startup. Tracked as HXC-234.
- **RAG silently degrades to a non-semantic embedder.** The gateway warns that `HELIX_EMBEDDING_PROVIDER` is unset so it is using the `HashEmbedder`, which by its own warning text does “NOT capture semantic similarity”. Retrieval keeps answering rather than failing, so degraded results are indistinguishable from good ones at the call site. Tracked as HXC-235.
- **The gateway runs Gin in debug mode** under `systemd` (`[WARNING] Running in "debug" mode. Switch to "release" mode in production.`). Tracked as HXC-229.
- **QUIC receive-buffer warning** from `helixllm-gateway` (failed to sufficiently increase receive buffer size (was: 208 kiB, wanted: 7168 kiB, got: 416 kiB)) — a host `sysctl` tuning matter (`net.core.rmem_max`), not a service defect. Revision 2 recorded this as *not re-observed* and therefore unconfirmed; it **was re-observed on the 2026-08-06 boot**, so it is now confirmed present.

Resolved since Revision 2

- **The gateway's tracked unit template has caught up with its installed copy.** Revision 2 recorded that the two `Environment=` fixes (`HELIX_REDIS_HOST/PORT`, `HELIX_MODELS_DIR`) were present only in the installed unit and would be silently reverted by the next `install_systemd_units.sh` run. **That gap is closed** — verified 2026-08-06 that the tracked template now carries all nine `Environment=` lines identically, including `HELIX_REDIS_PORT=6380` and `HELIX_MODELS_DIR=%h/models`, and that the inert `HELIX_CACHE_REDIS_*` names are gone. The remaining diff is comment wording only; the units are functionally identical, and all five other units are byte-identical:

```
diff <(sed "s|@HELIX_ROOT@$PWD|g; s|@HELIXLLM_BIN@|
    $HOME/.local/bin/helixllm|g" \
    scripts/systemd/helixllm-gateway.service | grep -vE
    '^\\s*#|^\\s*$') \
    <(grep -vE '^\\s*#|^\\s*$' ~/.config/systemd/user/
    helixllm-gateway.service)
# → no output
```

The Environment=name-mismatch fixes described under *Gotchas* are therefore now reproducible from a fresh clone.

- **Redis reaches the gateway.** Following from the above, the 2026-08-06 boot logged KV cache: Redis connected rather than Revision 2's dial tcp 127.0.0.1:6379: connect: connection refused/falling back to in-memory. The HELIX_REDIS_PORT=6380 fix is confirmed working at runtime.

Resolved since Revision 1

- **LLMsVerifier now publishes real scores.** Revision 1 recorded GET :8100/api/scores returning {"scores":{}} (“nothing measured yet”). As of 2026-07-28 it returns {"scores":{"llamacpp":{"total":96,"model_count":1}}} — the verifier seeds its providers from the consumer-owned config/llmsverifier/config.yaml and has published its first real aggregate, so the gateway can adopt it instead of its static fallback table. Coverage is one provider so far; the remaining providers stay unscored until a verification pass runs with their API keys configured.

Sources verified 2026-08-06 (Revision 3)

PRIMARILY INTERNALLY DERIVED, as in Revision 2. What distinguishes this pass: Revision 2 explicitly declined to execute any systemctl start/stop, so its enable/disable and start-ordering claims were read from unit files rather than observed. **Revision 3 executed a full stop/start cycle** and every claim added above is from that captured run. Verified in this pass:

- **Target propagation, both directions**, by executing systemctl --user start helix.target (target went inactive→active with services already up) then stop (all six PartOf= members went inactive, 0 containers, all 18 ports closed) then start again from cold.
- **PartOf=helix.target on all six services**, by systemctl --user show -p PartOf — this is what makes the documented “stop brings it all down” true, and WantedBy= alone would not.

- **The cold-boot helixagent failure**, from the boot-window journal (- - after - cursor) showing the fatal Cognee check, the systemd restart, and the passing retry, corroborated by `NRestarts=1`.
- **The 37-minute gateway listener gap**, from its journal timestamps (`ExecStart` → server listening) cross-checked against a `curl` returning HTTP 000 mid-window and `ss -ltn` showing 8443 closed at that moment.
- **The missing tables**, by `to_regclass()` against all three databases directly, not merely from the error text.
- **llama-server absence**, by command `-v` plus a bounded find over the unit's PATH directories and \$HOME.
- **Template drift closure**, by expanding all six templates and diffing them against the installed copies.
- **Runtime serving**, by HTTP GETs to all four documented health routes plus `/v1/models` and `/v1/hardware`, a live `psql` query, a Redis PING/SET/GET/DEL round-trip, and six infra HTTP probes — all 200 or their protocol equivalent.

Honest boundary (§11.4.6 / §11.4.99(B)): (1) the upstream systemd manual pages were again NOT re-fetched, so generic systemd semantics still carry no external-currency claim — but the propagation and job-ordering behaviour described above is now observed rather than inferred; (2) `enable/disable` were still not exercised (the units were already enabled and deliberately left that way), so those commands remain documented from [Install] sections; (3) boot-at-host-boot via `linger` was not tested — that needs a host reboot, which is forbidden under CONST-033 / §11.4.133, so the `linger` chain is verified only as far as `Linger=yes` and the two symlink sets; (4) the three postgres FATAL: role/database does not exist lines in the 2026-08-06 journal are the verifying agent's own mis-typed `psql` probes at 23:26/23:37/23:47 local, not platform defects, and are excluded from the findings above.

Evidence: `docs/qa/live_systemd_boot_20260806T145857Z/`.

Sources verified 2026-07-28 (Revision 2)

PRIMARILY INTERNALLY DERIVED — this document describes THIS repository's own systemd user-unit installation, so its authoritative sources are the tracked unit definitions, the installer, and the consuming binaries' own configuration code. Verified in this pass:

- **The six installed units** at `~/.config/systemd/user/` read directly: `helixcode-infra`, `llmsverifier`, `helixllm-coder`, `helixllm-gateway`, `helixagent`, `helixcode-server`. The `Requires=` / `Wants=` / `WantedBy=helix.target` claims above are quoted from those files.
- **The boot chain**, by listing `default.target.wants/` (contains `helix.target`), `helix.target.wants/` (contains all six services), and `loginctl show-user -p Linger(Linger=yes)`.

- **Env-tag claims**, by `grep -n 'env:'` over `submodules/helix_llm/internal/shared/config/config.go`: `HELIX_MODELS_DIR` default / `models` at line 70; `HELIX_REDIS_HOST` line 121; `HELIX_REDIS_PORT` default 6379 line 122. The `HELIX_CACHE_REDIS_*` names the template still sets appear nowhere in that file.
- **The gateway health route**, by locating its registration at `submodules/helix_llm/internal/server/server.go:167` (`s.engine.GET("/internal/health", ...)`), and confirming `/v1/models` is the *downstream-provider* probe path (`internal/a2a/downstream.go:72`, `internal/mcpgateway/downstream.go:69`).
- **The port map**, against the ports: entries of `compose.helixcode-infra.yml` (including weaviate's second mapping 50051, previously undocumented) and the 11 services it defines.
- **The template drift**, by expanding `scripts/systemd/helixllm-gateway.service` and diffing it against the installed copy.
- **Runtime state**, by read-only HTTP GETs (`/api/scores`, `/internal/health`, two `/health` routes — all 200) and a read-only `journalctl --user` window for the `distributed_locks` gap.

Honest boundary (§11.4.6 / §11.4.99(B)) — three explicit gaps: (1) the upstream `systemd` manual pages (`systemd.unit`, `systemd.service`, `systemd.target`, `journalctl`) were NOT re-fetched in this pass, so the generic `systemd` semantics described here rest on the in-repo unit files alone and carry no external-currency claim; (2) no `systemctl start/stop/restart/enable` was executed in this pass — the platform was already running and was deliberately not mutated, because other work is live on this shared host (§11.4.174), so the `enable/disable` commands above are documented from the units' [Install] sections and the installer, not from an executed `enable` cycle; (3) the QUIC receive-buffer warning is carried over from an earlier run and was not re-observed, as flagged in *Known gaps*. Re-verify against the official `systemd` documentation per the §11.4.99 180-day staleness window before this guide is used as external-facing operator authority.